Aprendizaje Automático · Maestría en Ciencia de Datos y Analítica · Universidad EAFIT

Retrieval-Augmented ML Systems

Modelos que consultan una memoria externa antes de responder

Santiago Betancur - Santiago Vélez

Motivación

Los límites de lo que sabe un LLM

Si le preguntas a un LLM por un lenguaje de nicho, *¿cómo sabes si la respuesta es cierta o solo suena cierta?*

¿Qué significa el código de error T-24 que salió en el tablero de mi moto **Velora**?

Por lo general responderá con seguridad, dando a entender que sabe la respuesta y aún así entregar una respuesta errónea.

Conocimiento congelado
Lo que cambió después del corte de entrenamiento no existe para el modelo.

Sin acceso a datos privados
Manuales internos y datos de la empresa nunca hicieron parte de su entrenamiento.

Alucinaciones
Ante un vacío genera una respuesta plausible y segura, aunque sea falsa.

Sin trazabilidad
No puede decir de dónde salió una respuesta, así que es difícil auditarla.

Motivación

La idea: consultar antes de responder

Memoria paramétrica en los

Guardan el lenguaje, el razonamiento y patrones generales. Cambiarlos implica reentrenar o hacer fine-tuning.

Memoria no paramétrica

Un corpus indexado D

Guarda hechos concretos que se consultan en cada pregunta. Se actualiza agregando o borrando documentos.

El corpus define **qué sabe** el sistema; el modelo define **cómo razona** con eso. Resultado: actualizar sin reentrenar, usar datos privados y citar fuentes.

Fundamentos · probabilidad

La formulación probabilística

$$p(y \mid x) \approx \Sigma\, p\eta(z \mid x) \cdot p\theta(y \mid x, z)$$

la suma recorre solo los k documentos más relevantes: z ∈ top-k(x)

x, y
La pregunta de entrada y la respuesta generada.

z
Documento recuperado: variable latente que se marginaliza.

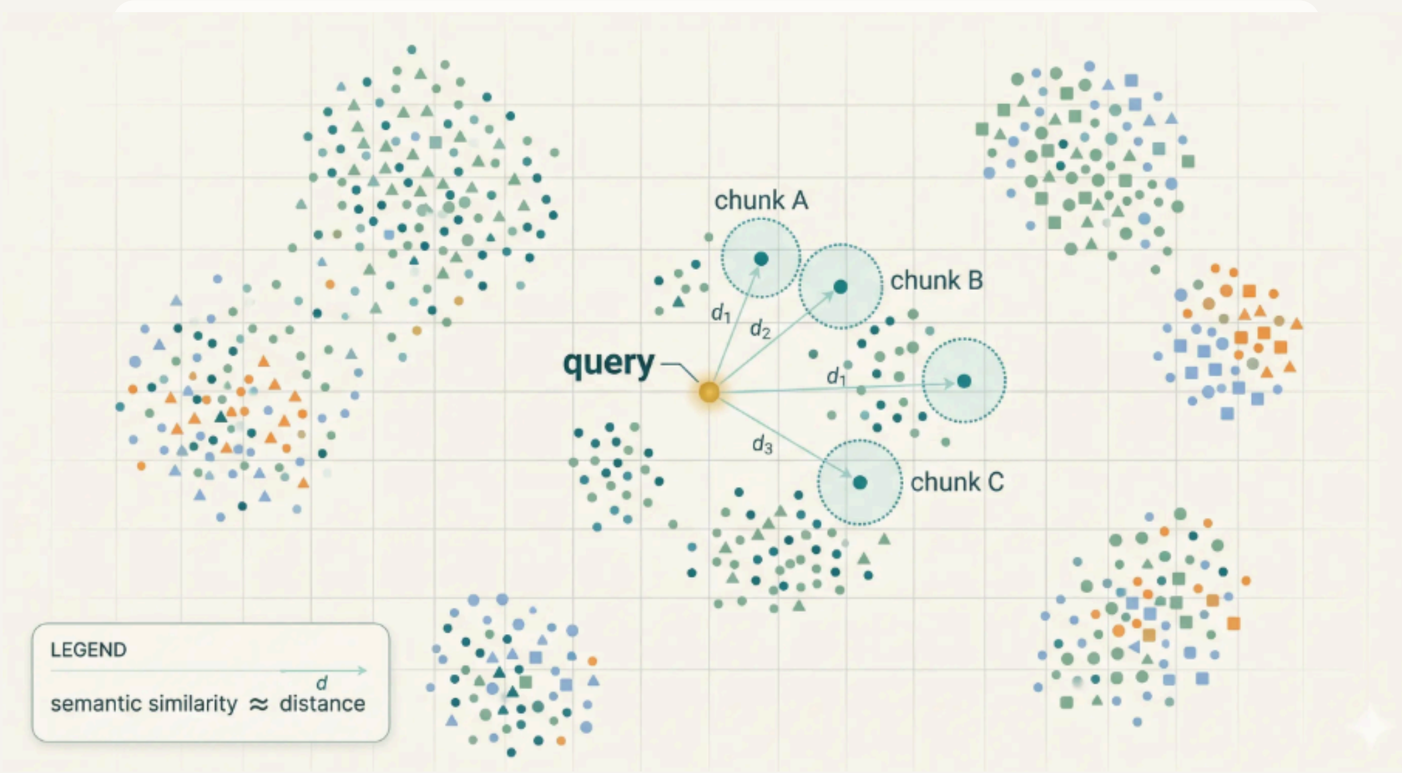
$p\eta(z \mid x)$
El retriever: qué tan relevante es z para la pregunta.

$p\theta(y \mid x, z)$
El generador: la respuesta condicionada a x y a z.

RAG-Sequence: los mismos k documentos para toda la respuesta. **RAG-Token**: puede apoyarse en un documento distinto en cada token.

En la práctica (LLM + prompt)
No se marginaliza: se concatenan los top-k en el prompt y el LLM estima pθ(y | x, $z_1 \ldots z_k$). La calidad depende de que pη ponga arriba el z correcto.

Recuperar es buscar vecinos cercanos



Búsqueda exacta: kNN

kNN(q) = los k vectores d ∈ D con mayor sim(q, d)

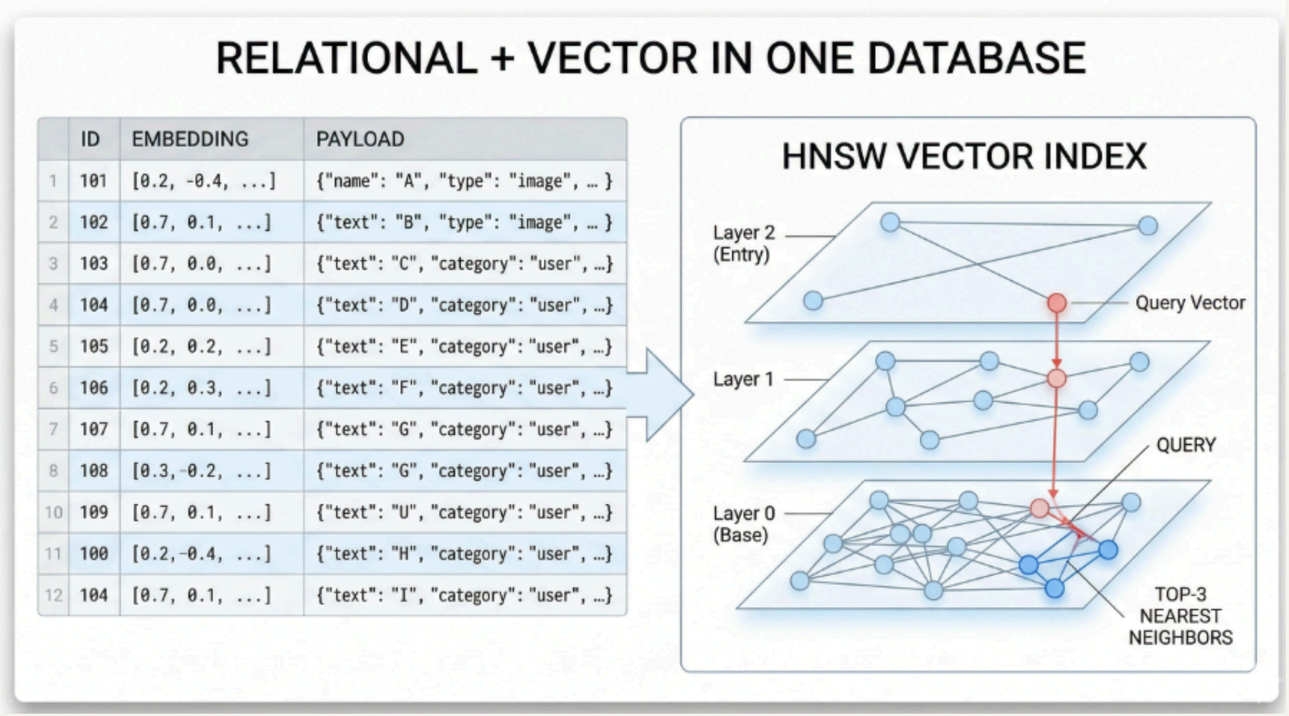
Medida	Fórmula
Coseno	$q \cdot d / (\|q\| \, \|d\|)$
Producto punto	$q \cdot d = \Sigma \, q_i \, d_i$
Euclidiana (L2)	$\|q - d\| = \sqrt{\Sigma \, (q_i - d_i)^2}$

Costo: O(N · d) por consulta

1 M fragmentos × 384 dimensiones ≈ 384 M operaciones por pregunta.

Fundamentos · algoritmos

ANN: búsqueda aproximada a escala



HNSW · grafo jerárquico
Capas de grafos de vecinos: búsqueda voraz de lo grueso a lo fino, ≈ O(log N). Rápido y con recall alto; usa más memoria.

IVF · particiones con k-means
Agrupa los vectores en celdas y solo revisa las más cercanas a la pregunta. Más celdas revisadas = más recall y más latencia.

PQ · cuantización por productos
Parte el vector en m subvectores y guarda un código de 1 byte por cada uno: 1536 floats (6 KB) → 96 bytes con m = 96.

Tres estrategias: sparse, dense e hybrid

Sparse · léxica

BM25: palabras exactas

Vector de |V| dimensiones, casi todo ceros: un peso por término. Índice invertido.

Fuerte: nombres de funciones, códigos, siglas.

Débil: sinónimos y paráfrasis.

Dense · semántica

Bi-encoder: significado

Vector de 384 a 1536 dimensiones, todas con valor. Dos encoders y un producto punto.

Fuerte: paráfrasis, preguntas conceptuales, multilingüe.

Débil: identificadores raros que el modelo no vio.

Hybrid · ambas

Fusión de rankings

Corre las dos búsquedas y combina las listas resultantes.

Fuerte: corpus técnicos con ambos tipos de pregunta.

Débil: más piezas que ajustar (α, k, normalización).

Aplicaciones

La recuperación como patrón general

$$\hat{y} = f(\, x,\, R(x;\, D)\,)$$

Cualquier modelo puede recibir, además de la entrada x, los vecinos que el retriever R encuentra en una memoria D.

LLMs generativos

RAG. Fragmentos recuperados en el prompt: chatbots, asistentes, búsqueda con respuesta.

kNN-LM. Mezcla la predicción del LM con la de sus vecinos más cercanos.

RETRO. Recupera de 2 billones de tokens; iguala a GPT-3 con 25× menos parámetros.

Clasificadores

k-NN. La forma más antigua: votar la clase de los k ejemplos más parecidos.

TabR. En datos tabulares, atiende a las filas de entrenamiento más similares.

RA-CLIP. En visión, recupera pares imagen-texto para clasificar mejor.

Otros predictivos

Recomendación. Generar candidatos por similitud y luego rankearlos.

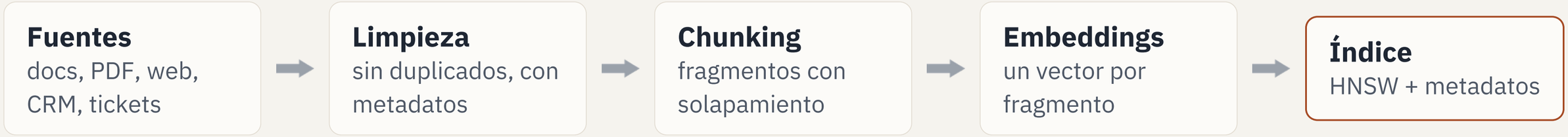
Anomalías. La distancia a los vecinos más cercanos es el score.

Pronóstico. Recuperar episodios históricos parecidos como contexto.

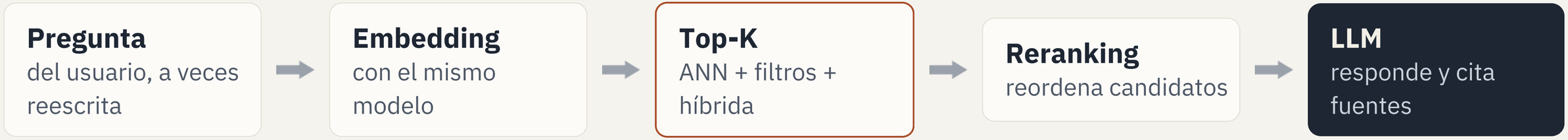
Preparación del sistema

El pipeline de un sistema RAG

Fase offline · indexación — al cargar o cambiar documentos



Fase online · consulta — en cada pregunta



La base de conocimiento es el techo del sistema: si el dato no está, está duplicado o desactualizado, ningún retriever lo arregla. Curar, versionar y re-indexar solo lo que cambia.

Preparación del sistema

Categorización vectorial: tres decisiones

1 Chunking

- **Fijo** con solapamiento: simple y predecible.
- **Recursivo** por estructura: títulos, párrafos, bloques de código.
- **Semántico**: corta donde cambia el tema.
- Partida: 200–1000 tokens, 10–20 % de solapamiento.

2 Embeddings

- El **mismo modelo** para fragmentos y preguntas.
- Dimensión: 384 en multilingual-e5-small, 1536 en text-embedding-3-small.
- Normalizar a norma 1: coseno = producto punto.
- Cambiar de modelo obliga a re-indexar todo.

3 Índice

- **HNSW en pgvector**: m = 16, ef_construction = 64 por defecto.
- ef_search se ajusta en cada consulta.
- Metadatos junto al vector: fuente, sección, fecha, permisos.
- Relacional + vectorial en una sola base.

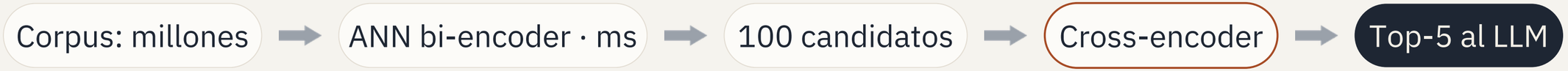
Chunk pequeño: preciso, pero pierde contexto. ← tamaño del chunk → **Chunk grande:** más contexto, señal diluida y más tokens.

Evaluación y ajuste

La parametrización del retrieval

Variable	Qué cambia al moverla	Punto de partida
Top-K	↑ K: más recall, menos precisión; más tokens, costo y latencia. Riesgo de «lost in the middle».	3–10 al LLM
Similitud y umbral	La métrica define la geometría. Un umbral alto quita ruido, pero puede dejar al LLM sin contexto.	Coseno normalizado; umbral calibrado
Filtros	Restringen por fuente, fecha o permisos: más precisión y seguridad. Filtrar después del ANN puede devolver menos de K.	Pre-filtrar siempre por permisos
Híbrida (α)	α → 1: solo denso; α → 0: solo BM25. RRF evita calibrar escalas distintas.	RRF con k = 60
Reranker	Un cross-encoder lee pregunta y fragmento juntos: sube MRR y nDCG a cambio de latencia.	50–100 candidatos → 5–10

Recuperación en dos etapas



Aplicación · demo

Sistema RAG para la consulta de códigos de errores en Motos

1

Corpus

Información sobre todos los códigos de errores documentados para todos los modelos de motos

2

Tokenización

El texto se vuelve tokens y luego números

3

Embedding

Vectores de 384 dimensiones con multilingual-e5-small

4

Similitud

Matriz de cosenos entre los fragmentos

5

Mapa en 2D

Proyección de los vectores con PCA y UMAP

6

Clasificar

Recuperar los K-TOP embeddings con mayor similitud

7

Comparativo

Comparación entre las 3 estrategias para el Retrieval

8

Evaluación - Reentrenamiento

¿Qué sucede si se pregunta por un código que no se encuentra en la base de conocimiento?

Qué observar: textos parecidos quedan cerca en el mapa, y BM25 y la búsqueda densa no siempre devuelven el mismo top-K.

Notebook en Google Colab

Cierre

Muchas Gracias.

Referencias

Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.

Guu et al. (2020). REALM: Retrieval-Augmented Language Model Pre-Training.

Karpukhin et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering.

Khandelwal et al. (2019). Generalization through Memorization: Nearest Neighbor Language Models.

Izacard y Grave (2020). Leveraging Passage Retrieval with Generative Models (FiD).

Borgeaud et al. (2021). Improving Language Models by Retrieving from Trillions of Tokens.

Izacard et al. (2022). Atlas: Few-shot Learning with Retrieval Augmented Language Models.

Asai et al. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique.

Robertson y Zaragoza (2009). The Probabilistic Relevance Framework: BM25 and Beyond.

Yan et al. (2024). Corrective Retrieval Augmented Generation.

Edge et al. (2024). From Local to Global: A Graph RAG Approach.

Malkov y Yashunin (2016). Efficient and Robust Approximate Nearest Neighbor Search Using HNSW Graphs.

Johnson et al. (2017). Billion-Scale Similarity Search with GPUs (FAISS).

Cormack et al. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods.

Liu et al. (2023). Lost in the Middle: How Language Models Use Long Contexts.

Es et al. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation.

Gorishniy et al. (2023). TabR: Tabular Deep Learning Meets Nearest Neighbors.

Gao et al. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey.